

```

1) public class A {
    private static int k = 0;
    private int i;

    public A(int j){
        i = j;
    }

    public void m1 (int i){
        A aux = new A(0);
        aux.m2(i);
        k = k + i + 1;
    }

    public void m2 (int i){
        this.i += i;
        i = i * 2;
        k = k + i;
    }
}

```

Se ejecutan las siguientes instrucciones:

```

A a = new A(3);
a.m2(5);
a.m1(5);
a.m2(5);

```

Indique el valor de la variable de clase K. **K = 36**

```

2) public class A {
    private static int k = 0;
    private int i;
    public A(int j){
        i = j;
    }
    public void m1(int i) {
        A aux = new A(0);
        aux.m2(i);
        k = k+i+1;
    }
    public void m2(int i) {
        this.i += i;
        i = i*2;
        k = k+i;
    }
}

```

Se ejecutan las siguientes instrucciones:

```
A a = new A(2);  
a.m1(5);  
a.m1(1);  
a.m2(2);
```

Indique el valor de la variable de clase K. **K = 24**

```
3) public class A {  
    private static int k = 0;  
    private int i;  
    public A(int j){  
        i = j;  
    }  
    public void m1(int i) {  
        A aux = new A(0);  
        aux.m2(i);  
        k = k+i+1;  
    }  
    public void m2(int i) {  
        this.i += i;  
        i = i*2;  
        k = k+i;  
    }  
}
```

Se ejecutan las siguientes instrucciones:

```
A a = new A(4);  
a.m2(2);  
a.m2(4);  
a.m1(2);
```

Indique el valor de la variable de clase K. **K = 19**

4) Implementar getter y setter

```
/* Si el argumento base es valioso, setea el valor y retorna  
true.  
* Si el valor es invalido, ignora el valor y retorna false.  
*@param base. La base valida debe ser un numero entre 0 y  
500.  
*/
```

```
public boolean setBase (int base){  
    if (base < 0 || base > 500){  
        return false;  
    }  
    else {
```

```

        this.base = base;
        return true;
    }

}

//Retorna el valor del radio
public int getBase () {
    return base;
}

```

5) Implemente getter y setter de la variable especificada

```

public class Docente {
    private int dni;
    //Constructor
    public Docente (int dni){
        this.dni = dni;
    }
    /**
     * si el argumento dni es valido, setea el valor y retorna
     true
     * Si el valor es invalido, ignora el valor y retorna false
     * @param dni. El dni valido debe ser un numero entre 1 y
     99999999
     */
    public boolean setDni(int dni){
        if(dni >= 1 && dni <= 99999999){
            this.dni = dni;
            return true;
        }
        else{
            return false;
        }
    }

    /**
     * Retorna el valor de dni
     */
    public int getDni () {
        return dni;
    }
}

```

6) Implemente el getter y el setter de acuerdo a la documentación de la clase

```

public class Rectangulo {
    public String identificador;
    /**

```

```

* Si el argumento es distinto de null, setea el
identificador,
* removiendo antes cualquier espacio inicial y convirtiendo
a minusculas.
* Si el argumento es null, ignora el valor sin setear el
identificador
* @param el nuevo identificador
*/
public void setIdentificador (String id){
    if (id!=null) {
        identificador=id.trim().toLowerCase();
    } //ignoro si es null
}
/**
* retorna el valor del identificador de este Circulo
*/
public String getIdentificador () {
    return identificador;
}
}

```

- 7) Implementar método `parcharTodas()` asumiendo que la clase `Bicicleta` tiene un método `public void parchar()`

```

import java.util.List;
public class Evaluacion {

    public HashMap <String, Bicicleta> clientes;

    //constructor
    public Evaluacion (){
        clientes = new HashMap <String, Bicicleta>();
    }

    //Recorre la coleccion de clientes y llama al metodo parchar
    de cada bicicleta.
    public void parcharTodas(){
        for (String key : clientes.keySet()){
            Bicicleta a = clientes.get(key);
            a.parchar();
        }
    }
}

```

- 8) En la clase especificada, implementar `public void pintarUnAuto (String color, String patente.`

Nota: Asuma que la clase Auto implementa el método: public void pintar (String color).

```
import java.util.HashSet;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;

public class Evaluacion {
    //mapa que relaciona Autos con una patente del tipo String
    public HashMap<String,Auto> mapaDeAutos;
    //Constructor
        public Evaluacion () {
            mapaDeAutos=new HashMap<String,Auto>();
        }

    //Pinta el auto con la patente indicada del color
    especificado
        public void pintarUnAuto (String color, String patente){
            for(String key:mapaDeAutos.keySet()){
                if(key.equals(patente)){
                    mapaDeAutos.get(key).pintar(color);
                }
            }
        }
    }
}
```

- 9) Para la clase especificada, implementar public void parcharTodas ().

Nota: Asuma que la clase Bicicleta implementa el método public void parchar ().

```
import java.util.HashSet;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;

public class Evaluacion {
    public HashMap<String,Bicicleta> clientes;
    //Constructor
        public Evaluacion () {
            clientes = new HashMap<String,Bicicleta>();
        }

    // Recorre la coleccion clientes y llama al metodo parchar de
    //cada bicicleta
        public void parcharTodas () {
            for (String key: clientes.keySet()){
                Bicicleta a = clientes.get (key);
                a.parchar();
            }
        }
    }
}
```